Functions in Python — A Self-Study Guide

Work through this at your own pace. Read each section, type out the code yourself, and predict outputs before you run them. Everything here is Python 3, and every example has been run, so the outputs shown are exactly what you'll get.

Contents: 1. What is a function · 2. Defining and calling · 3. Parameters and arguments · 4. Return vs print · 5. Default and keyword arguments · 6. Scope · 7. Functions calling functions · 8. Common mistakes · 9. Self-check · 10. Assignment

---

1. What is a function?

A function is a named block of code that does one job. You write it once, give it a name, and then "call" it whenever you need that job done — instead of copying the same lines again and again.

Analogy — a chai recipe. Imagine you write down a recipe called "make chai" once. After that, you don't re-explain the whole process every morning; you just say "make chai." You can even say "make chai with 2 spoons sugar" — same recipe, different input. A function works the same way: define the steps once, then call it with different inputs whenever you want.

Why functions matter:

- **No repetition —** write the logic once, reuse it everywhere.
- **Readable —** `calculate_gst(price)` tells you what's happening at a glance.
- **Easy to fix —** if there's a bug, you fix it in one place, not twenty.
- **Divide the work** — big programs become small, named pieces you can build and test one at a time.

---

2. Defining and calling a function

You **define** a function with the `def` keyword, and you **call** it by writing its name followed by `()`.

```python
def greet(name):              # define: 'name' is the input
    print("Namaste,", name + "!")

greet("Priya")                # call it
```

Output:

```text
Namaste, Priya!
```

Let's name each part:

- `def` — the keyword that starts a function definition.
- `greet` — the function's **name** (you choose it; use a clear, descriptive name).
- `(name)` — the **parameter** list: the inputs the function expects.
- `:` — ends the definition line.
- The **indented** lines below are the function **body** — the actual work. Indentation is how Python knows which lines belong to the function.

> [!IMPORTANT] Defining a function does **not** run it. The body only runs when you **call** the function, e.g. `greet("Priya")`. You can define once and call many times.

```python
greet("Rohan")
greet("Saanvi")
```

Output:

```text
Namaste, Rohan!
Namaste, Saanvi!
```

3. Parameters and arguments

A **parameter** is the name in the function definition (the placeholder). An **argument** is the actual value you pass when you call it.

```python
def add(a, b):            # a and b are PARAMETERS
    return a + b

print(add(40, 25))        # 40 and 25 are ARGUMENTS
```

Output: `65`

A function can take any number of parameters, including none:

```python
def area_rectangle(length, breadth):
    return length * breadth

print(area_rectangle(12, 5))   # 60
```

> [!NOTE] By default, arguments are matched to parameters **by position**: the first argument fills the first parameter, the second fills the second, and so on. In `area_rectangle(12, 5)`, `length` becomes `12` and `breadth` becomes `5`. (Section 5 shows how to pass them by name instead.)

---

4. Return versus print

This is the single most important idea about functions, and the one beginners most often get wrong.

- `print()` **shows** a value on the screen. That's all it does.

- `return` **hands a value back** to whoever called the function, so it can be stored or used in further calculations.

Compare these two:

```python
def square_print(n):
    print(n * n)          # only displays

def square_return(n):
    return n * n          # hands the value back
```

```python
result = square_print(5)      # screen shows 25
print(result)                 # None  ← nothing was returned!

value = square_return(5)      # nothing shown yet
print(value)                  # 25
print(value + 1)              # 26   ← we can use it in a calculation
```

Output:

```text
25
None
25
26
```

> [!IMPORTANT] If a function has no `return` statement (or a bare `return`), it hands back the special value `None`. So `square_print` returns `None` — you can see `25` on screen, but you cannot use it in a calculation. When you need the result for further work, `return` **it**.

A function can return more than one value (as a tuple), which you can unpack:

```python
def min_max(marks):
    return min(marks), max(marks)

lowest, highest = min_max([78, 92, 65, 88])
print(lowest, highest)        # 65 92
```

Returning `None` on purpose. Sometimes "no answer" is the right answer:

```python
def find_topper(marks):
    if not marks:             # empty list → no topper
        return None
    return max(marks)

print(find_topper([88, 95, 72]))   # 95
print(find_topper([]))             # None
```

---

5. Default and keyword arguments

Default arguments let a parameter have a fallback value, used when the caller doesn't supply one.

```python
def gst_price(price, rate=18):        # rate defaults to 18%
    return price + price * rate / 100

print(gst_price(1000))        # 1180.0   (uses rate = 18)
print(gst_price(1000, 5))     # 1050.0   (rate = 5 given)
```

Output:

```text
1180.0
1050.0
```

> [!NOTE] Parameters **with** defaults must come **after** parameters without defaults in the definition. `def f(a, b=10)` is valid; `def f(a=10, b)` is an error.

Keyword arguments let you pass values by **name** instead of position. This makes calls clearer and means order doesn't matter:

```python
def book_ticket(name, source, destination):
    return f"{name}: {source} to {destination}"

# passed by name — order doesn't matter
print(book_ticket(destination="Mumbai", name="Rohan", source="Pune"))
```

Output:

```text
Rohan: Pune to Mumbai
```

Taking many values with `*args`. When you don't know how many values will come, `*args` collects them all into one tuple:

```python
def total_bill(*items):        # any number of prices
    return sum(items)

print(total_bill(40, 25, 120, 15))   # 200
print(total_bill(50, 50))             # 100
```

Output:

```text
200
100
```

---

6. Scope: where a variable lives

A variable created **inside** a function is **local**: it exists only while the function runs, and the outside world can't see it. A variable created outside is **global**.

```python
x = 10                  # global

def change():
    x = 99              # a NEW local x, separate from the global one
    return x

print(change())         # 99  (the local x)
print(x)                # 10  (the global x is untouched)
```

Output:

```text
99
10
```

> [!IMPORTANT] Assigning to `x` inside the function creates a **local** variable; it does **not** change the global `x`. This is a feature, not a bug — it keeps functions from accidentally stepping on each other's variables. Each call works in its own little room.

A function *can* read a global value, but to be safe and clear, prefer **passing values in as arguments** and **returning results out**, rather than relying on globals.

---

7. Functions calling functions

Functions become powerful when they work together: one function can call another. You build small, trustworthy pieces and combine them.

```python
def is_even(n):
    return n % 2 == 0           # True if n is even

def count_even(numbers):
    count = 0
    for n in numbers:
        if is_even(n):          # reuse the small function
            count += 1
    return count

print(count_even([3, 8, 12, 7, 4]))   # 3
```

Output: `3`

Here `count_even` doesn't re-check "is this even?" itself — it **trusts** `is_even` to do that one job. This is how real programs are built: small helpers, combined.

---

8. Common mistakes to avoid

Mistake 1 — using `print` where you need `return`.

```python
# BUG: this only shows the result; it can't be used later
def double(n):
    print(n * 2)

answer = double(10)       # screen shows 20
print(answer * 3)         # ERROR / nonsense: answer is None, not 20
```

Fix: `return n * 2`, then `answer` holds `20` and `answer * 3` works.

Mistake 2 — forgetting to call the function.

```python
def greet(name):
    print("Hello,", name)

greet                    # BUG: this does nothing — no parentheses!
greet("Aarav")           # correct: the () actually calls it
```

Writing just `greet` refers to the function but never runs it. You need `greet("Aarav")`.

Mistake 3 — wrong number of arguments.

```python
def add(a, b):
    return a + b

add(5)                   # BUG: TypeError — b is missing
```

A function expects exactly as many arguments as it has parameters (unless some have defaults). Give `add` both values: `add(5, 3)`.

Mistake 4 — expecting a function to change a global variable by assigning inside it.

```python
score = 0
def add_point():
    score = score + 1    # BUG: tries to read a local 'score' that doesn't exist yet
add_point()              # UnboundLocalError
```

The clean fix is to pass the value in and return the new one:

```python
def add_point(score):
    return score + 1

score = 0
score = add_point(score)   # now score is 1
```

---

9. Self-check

Try each before opening the answer. Predict the output or write the code on paper first.

Self-check A. What does this print?

```python
def mystery(x):
    return x * x + 1

print(mystery(3))
```

▶ **Answer**

Self-check B. What is the value of `result` after this runs, and why?

```python
def show_sum(a, b):
    print(a + b)

result = show_sum(4, 6)
```

▶ **Answer**

Self-check C. Write a function `greet_city(name, city)` that returns a sentence like `"Hello Ravi from Delhi"`. Then call it using **keyword arguments** in any order.

▶ **Answer**

---

10. Assignment

Solve all 10. For "write a function" questions, make sure you use `def`, the right parameters, and `return` where a value is needed. For "output" questions, work out the answer by hand before running anything.

Easy

Q1. What is the exact output of this program?

```python
def wish(name):
    print("Happy Diwali,", name)

wish("Aarav")
```

Q2. Write a function `cube(n)` that returns the cube of a number. Example: `cube(4)` should return `64`.

Q3. Using the function below, what do `price_with_tax(100)` and `price_with_tax(100, 25)` each return? Explain why they differ.

```python
def price_with_tax(amount, tax=10):
    return amount + tax
```

Q4. After this program runs, what value does `r` hold? Explain in one sentence.

```python
def show(n):
    print(n + 1)

r = show(5)
```

Medium

Q5. What is the exact output of this program?

```python
def intro(name, city):
    return name + " from " + city

print(intro(city="Jaipur", name="Diya"))
```

Q6. Write a function `average(a, b, c)` that returns the average of three numbers. Example: `average(80, 90, 100)` should return `90.0`.

Q7. What is the exact output of this program? Explain why the global `y` has the value it does at the end.

```python
y = 5

def update():
    y = 20
    return y

print(update(), y)
```

Q8. Write a function `divide(a, b)` that returns **two** values: the quotient and the remainder when `a` is divided by `b`. Then show how to call it so that the quotient is stored in `q` and the remainder in `r`. Example: for `divide(17, 5)`, `q` should be `3` and `r` should be `2`.

Hard

Q9. Write **two** functions:

- `is_vowel(ch)` — returns `True` if a single character is a vowel (a, e, i, o, u), ignoring case.
- `count_vowels(word)` — uses `is_vowel` to count how many vowels are in a word.

Your `count_vowels` must **call** `is_vowel` (do not rewrite the vowel check inside it). Example: `count_vowels("BHARAT")` should return `2`.

Q10. Write a function `simple_interest(principal, rate, time)` that returns the simple interest, calculated as `(principal × rate × time) / 100`. Example: `simple_interest(15000, 8, 3)` should return `3600.0`. Then answer: if someone calls `simple_interest(15000, 8)` (with only two arguments), what will happen and why? How would you change the function so that `time` defaults to `1` year when it isn't given?